

HTTP, formularios, cookies y sesiones

Objetivos

Leer una petición y una respuesta, localizar entradas controlables, distinguir transporte de estado y predecir qué parser procesará el cuerpo.

Ciclo petición-respuesta

HTTP intercambia mensajes. El cliente envía método, objetivo, campos y, opcionalmente, cuerpo. El servidor selecciona un recurso, aplica autenticación y autorización, procesa la representación y devuelve estado, campos y cuerpo. HTTP es sin estado por diseño; una sesión es una capa de aplicación que relaciona peticiones mediante un identificador.

```
POST /login?next=/panel HTTP/1.1
Host: laboratorio.local
Content-Type: application/x-www-form-urlencoded
Cookie: session=abc123
```

```
user=ana&password=prueba
```

Entradas distintas: ruta /login, parámetro next, cabecera Host, cookie session y campos del cuerpo. No las procesa necesariamente el mismo componente.

Métodos, cabeceras y estados

- GET solicita una representación y suele transportar parámetros en la query.
- POST entrega datos para que el recurso los procese; no implica por sí solo crear algo.
- PUT, PATCH y DELETE expresan reemplazo, cambio parcial y eliminación, pero la aplicación decide qué autoriza.
- Content-Type declara cómo interpretar el cuerpo; Accept expresa qué respuesta prefiere el cliente.
- Un 2xx indica que HTTP completó la operación, no que la lógica sea segura.
- Un 3xx dirige al cliente; un 401 pide autenticación y un 403 rechaza una identidad ya evaluada. La implementación puede usarlos mal.
- Un 5xx puede revelar un error interno, pero no confirma vulnerabilidad.

Formatos de entrada

Formato	Separación	Parser habitual	Error útil
Query string	?a=1&b=2	Parser de URL/framework	parámetro ausente o duplicado
application/x-www-form-urlencoded	a=1&b=2	Parser de formulario	tipo o campo no válido
application/json	JSON	Deserializador JSON	posición y token inválidos
multipart/form-data	límites y partes	Parser multipart/upload	límite, nombre o fichero inválido

Cambiar solo el cuerpo sin cambiar Content-Type prueba una combinación distinta de la que crees. En Repeater conserva primero una petición válida y modifica una variable cada vez.

Cookies, sesiones y autenticación

La cookie es transporte de datos cliente-servidor. La sesión es el estado asociado, normalmente en servidor. La autenticación responde quién eres; la autorización, si esa identidad puede realizar la acción sobre ese objeto.

Preguntas mínimas:

- ¿El identificador cambia tras iniciar sesión o elevar privilegios?
- ¿Cerrar sesión invalida el estado del servidor o solo borra la cookie?
- ¿La autorización se verifica en cada objeto y acción?
- ¿Secure, HttpOnly y SameSite protegen el transporte esperado?
- ¿El cliente oculta una acción o el servidor realmente la rechaza?

Modelo mental

Bytes HTTP -> servidor/proxy -> parser HTTP -> router -> autenticación
-> parser del cuerpo -> lógica -> autorización -> dependencia -> respuesta

Una diferencia entre proxy y backend puede cambiar el significado. Antes de probar un payload, identifica cuántas capas lo decodifican.

Práctica segura

En una petición de laboratorio, cambia por separado query, cookie, cuerpo y método. Registra qué cambia en estado, longitud, cabeceras y contenido. Una diferencia es un indicio; confirma que depende de tu entrada y no de tiempo, caché o sesión.

Caso guiado: ¿sesión, caché o autorización?

Una petición autenticada devuelve el pedido 1042:

```
GET /api/orders/1042 HTTP/1.1
Host: shop.lab
Cookie: session=A7F1
```

Sin cookie responde 401. Con la cookie de otra cuenta responde 200, pero muestra el pedido 1042 de la primera cuenta.

Observación: la cookie es necesaria, pero dos identidades leen el mismo objeto. Qué sé realmente: hay autenticación; todavía no sé si falta autorización por objeto o si una caché sirve una respuesta anterior. Hipótesis: H1, el backend autoriza solo "usuario autenticado"; H2, una caché ignora la cookie. Experimento discriminatorio: cada cuenta solicita dos pedidos propios distintos añadiendo Cache-Control: no-cache y un parámetro inocuo único. Predicción: si H1 es cierta, la cuenta B seguirá leyendo 1042 y podrá consultar otros identificadores; si H2 es cierta, los marcadores o la cabecera alterarán el patrón. Resultado del escenario: B lee 1042 y recibe 403 al pedir un pedido inexistente, con o sin marcador. Conclusión: el router alcanza lógica de objeto y la sesión se reconoce, pero falta comprobar pertenencia. Siguiente paso: documentar el par identidad-objeto-acción; no intentar "romper la cookie".

Ejercicio de parser HTTP

La aplicación acepta este cuerpo:

```
POST /profile HTTP/1.1
Content-Type: application/json

{"name": "Ana"}
```

Al cambiar solo el cuerpo a name=Ana devuelve 400 con Unexpected token 'a' at position 1. La prueba siguiente no es buscar otro payload: es reconocer que el deserializador JSON rechazó el transporte antes de que name alcanzara la lógica. El control correcto es enviar JSON válido con un valor marcador y observar si cambia el perfil.

Resumen

HTTP mueve representaciones; no aporta por sí solo sesiones ni autorización. Cada ubicación de entrada tiene parser, normalización y frontera de confianza propios.

Referencias

- [RFC 9110: HTTP Semantics](#)
- [Conceptos internos de autenticación y Burp](#)

Siguiente: Redes, DNS y URL. Ejercicios: cuaderno de ejercicios.